UNIVERSIDAD DE LOS ANDES
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
ISIS2311L – REDES Y COMUNICACIONES

LABORATORIO 3

SOCKETS Y CAPA DE TRANSPORTE

Informe de Actividades y Preguntas de Análisis

GRUPO 2:

Luisa María Mora Pérez – 202410437
Juan Ángel Aicardy Pacheco – 202415378

2026-02

Índice

1. Implementación del Sistema y Documentación de Sockets (Actividades 1 y 2) **2**

2. Verificación de Envío de Mensajes y Capturas (Actividades 3 y 4) **3**
2.1. Verificación de Envío de Mensajes (Mínimo 10 por Publicador) 3
2.2. Archivos de Captura Wireshark (.pcap) 3

3. Tabla Comparativa de Desempeño: TCP vs. UDP (Actividad 5) **5**

4. Respuestas a las Preguntas de Análisis Objetivo (Actividad 6) **6**
4.1. 1. ¿Qué ocurriría si en lugar de dos publicadores hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del broker bajo TCP y bajo UDP? . . . 6
4.2. 2. Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario? 6
4.3. 3. En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica. 6
4.4. 4. Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia? 7
4.5. 5. Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2–1 y luego el 1–1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación? 7
4.6. 6. ¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto? 8
4.7. 7. ¿Qué sucede si el broker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión? 8
4.8. 8. ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué? 8
4.9. 9. Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró? . . 9
4.10. 10. Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una combinación de ambos? Justifique con base en lo observado en el laboratorio. 9

1. Implementación del Sistema y Documentación de Sockets (Actividades 1 y 2)

El sistema publicador–suscriptor para noticias deportivas en tiempo real se implementó en lenguaje C estándar sin librerías externas de abstracción de red. En cumplimiento del requerimiento, a continuación se documenta de manera detallada y punto a punto la interacción del programa con cada una de las funciones de la API estándar de sockets de Berkeley/POSIX (`sys/socket.h`, `netinet/in.h`, `arpa/inet.h`):

1. `socket(int domain, int type, int protocol)`:
   - **Propósito:** Solicita al kernel del sistema operativo la creación de un nuevo descriptor de archivo para comunicación de red.
   - **En UDP:** Se invoca como `socket(AF_INET, SOCK_DGRAM, 0)`. El parámetro `AF_INET` fija el dominio a IPv4 y `SOCK_DGRAM` configura el socket como no orientado a conexión basado en datagramas.
   - **En TCP:** Se invoca como `socket(AF_INET, SOCK_STREAM, 0)`. `SOCK_STREAM` configura un canal bidireccional fiable orientado a flujo de bytes continuo.
2. `bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)`:
   - **Propósito:** Asocia el descriptor de socket con una tupla local fija (dirección IP y puerto).
   - **Uso en el Broker:** Se emplea tanto en el Broker UDP como en el Broker TCP para fijar la escucha en el puerto `8080` y en la constante `INADDR_ANY` (todas las interfaces de red del equipo). Requiere el llenado previo de la estructura `struct sockaddr_in` y el cast genérico a `struct sockaddr*`.
3. `listen(int sockfd, int backlog)` **(Exclusivo TCP):**
   - **Propósito:** Marca el socket pasivo del Broker TCP como receptor de solicitudes de conexión entrantes, especificando en `backlog` la longitud máxima de la cola de conexiones pendientes antes de ser aceptadas.
4. `accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)` **(Exclusivo TCP):**
   - **Propósito:** Extrae la primera solicitud de conexión de la cola del socket en escucha, completa el *three-way handshake* y crea un **nuevo descriptor de socket dedicado** e independiente para la comunicación con ese cliente específico, preservando el socket original libre para aceptar nuevos clientes.
5. `connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen)` **(Exclusivo TCP):**
   - **Propósito:** Utilizado por los publicadores y suscriptores TCP para iniciar activamente el establecimiento de conexión hacia la dirección y puerto del Broker (`127.0.0.1:8080`), enviando el paquete inicial `SYN`.
6. `sendto()` **y** `recvfrom()` **(Exclusivos UDP):**
   - `sendto()`**:** Envía un datagrama autónomo especificando directamente la dirección y puerto de destino en cada llamada sin necesidad de conexión previa. Utilizado por los publicadores para enviar noticias al Broker y por el Broker para retransmitirlas a los suscriptores.

- `recvfrom()`: Función bloqueante que recibe el siguiente datagrama que arribe al socket y almacena automáticamente en una estructura `sockaddr_in` la dirección IP y puerto efímero del nodo que emitió el paquete, permitiendo al Broker UDP registrar nuevos suscriptores a partir de su mensaje `SUB:<tema>`.

7. `send()` y `recv()` **(Exclusivos TCP):**
   - **Propósito:** Transmiten y reciben datos sobre un socket TCP previamente conectado. Dado que TCP es un flujo continuo de bytes (*byte-stream*) sin delimitadores implícitos de mensaje, las aplicaciones delimitan cada noticia mediante un salto de línea (`\n`) para prevenir que múltiples mensajes se concatenen en una única lectura.
8. `htons()` y `inet_pton()`:
   - `htons()`: (*Host to Network Short*) Convierte el entero del puerto (8080) del orden de bytes de la máquina (*Little-Endian* en procesadores x86) al formato estándar de red (*Big-Endian*).
   - `inet_pton()`: Convierte la cadena legible de la dirección IP (`"127.0.0.1"`) a su formato binario en la estructura de red.

2. Verificación de Envío de Mensajes y Capturas (Actividades 3 y 4)

2.1. Verificación de Envío de Mensajes (Mínimo 10 por Publicador)

En las pruebas experimentales se ejecutaron simultáneamente 1 Broker, 2 Publicadores y 3 Suscriptores. Para verificar la recepción y el enrutamiento selectivo por temas, se emitieron exactamente 10 noticias desde cada publicador:

- **Publicador 1 (Tópico `futbol`, Puerto efímero 36437):** Emitió 10 noticias consecutivas: *"gol"*, *"de"*, *çolombia"*, *"de"*, *çolombia"*, *"decolooombiaa"*, *"1-0"*, *"falta"*, *.^{er}a mano omeomeome"*, *"sopa"*. Se comprobó que las 10 noticias llegaron únicamente a los dos suscriptores del tema fútbol (puertos 36835 y 43043).
- **Publicador 2 (Tópico `basket`, Puerto efímero 54648):** Emitió 10 noticias consecutivas: *"lowk"*, *"no "*, *"sen"*, *"nada"*, *"de"*, *"basket"*, *.^{en}tonces"*, *"voy a decir"*, *"1-0"*, *"20-0"*. Se comprobó que las 10 noticias llegaron únicamente al suscriptor del tema básquet (puerto 36980), sin fugas entre tópicos.

2.2. Archivos de Captura Wireshark (.pcap)

El tráfico completo de ambas versiones fue registrado en la interfaz de red y almacenado en los dos archivos solicitados:

1. `udp_pubsub.pcap`: Contiene los 50 datagramas del intercambio UDP (registrado en la carpeta local `capturas/udp_pubsub.pcap`).
2. `tcp_pubsub.pcap`: Contiene el tráfico de establecimiento de conexión (handshake), transmisión y desconexión TCP.

Ambos archivos de captura se encuentran disponibles para descarga en el siguiente enlace compartido:

Enlace OneDrive a Capturas: https://uniandes-my.sharepoint.com/:f:/r/personal/lm_morap1_uniandes_edu_co/Documents/Redes%20-%20Lab3%20-%20Grupo2

3. Tabla Comparativa de Desempeño: TCP vs. UDP (Actividad 5)

En la Tabla 1 se compara el desempeño de ambos protocolos bajo los cuatro criterios especificados en la guía de laboratorio:

Criterio	**TCP (Transmission Control Protocol)**	**UDP (User Datagram Protocol)**
Confiabilidad	**Alta (Garantizada por diseño):** Utiliza acuses de recibo (*ACKs*) y temporizadores de retransmisión (*RTO*). Si un paquete se descarta en la red, el emisor lo retransmite automáticamente hasta confirmar su recepción.	**Nula (Mejor esfuerzo / *Best Effort*):** No implementa mecanismos de acuse de recibo ni retransmisiones automáticas. Si un datagrama se pierde en un router congestionado, se pierde definitivamente a nivel de transporte.
Orden de entrega	**Garantizado:** Emplea números de secuencia de 32 bits. Si los segmentos llegan desordenados a la capa de red, el receptor los almacena en su buffer y los reordena antes de entregarlos en el orden exacto a la aplicación.	**No garantizado:** Cada datagrama es completamente independiente. Si toman rutas distintas o sufren retardos dispares, pueden arribar en un orden diferente al de emisión o llegar duplicados.
Pérdida de mensajes	**Transparente a la aplicación:** Las pérdidas son detectadas y resueltas internamente por el protocolo de transporte, asegurando que la aplicación reciba el flujo de datos completo y sin faltantes.	**Tolerada / Expuesta a la aplicación:** La capa de transporte no reporta pérdidas. Si la aplicación requiere que un mensaje no se pierda, debe implementar su propio protocolo de control en capa de aplicación.
Overhead de cabeceras	**Elevado (Mínimo 20 bytes por segmento):** Cabecera base de 20 bytes (hasta 60 bytes con opciones). Introduce además sobrecarga por paquetes puros de control: *handshake* de 3 vías (SYN), paquetes de confirmación ACK y cierre FIN.	**Mínimo (8 bytes fijos por datagrama):** Cabecera sumamente ligera con únicamente 4 campos de 2 bytes: puerto origen, puerto destino, longitud y checksum. No genera ningún paquete previo ni posterior de control.

Cuadro 1: Comparación de desempeño entre TCP y UDP según los criterios del laboratorio.

4. Respuestas a las Preguntas de Análisis Objetivo (Actividad 6)

4.1. 1. ¿Qué ocurriría si en lugar de dos publicadores hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del broker bajo TCP y bajo UDP?

- **Bajo TCP:** El Broker se vería forzado a mantener abiertas y activas al menos 100 conexiones TCP dedicadas exclusivamente a los publicadores (además de las de los suscriptores). Cada conexión consume descriptores de archivo en el sistema operativo y memoria del kernel para sus buffers de transmisión y recepción (`SO_RCVBUF` y `SO_SNDBUF`, típicamente entre 64 y 128 KB por socket). Además, el Broker requiere esquemas de multiplexación I/O (`select`, `poll` o hilos) con un elevado coste en cambios de contexto e inspección de descriptores, limitando drásticamente la capacidad de escala vertical del proceso.

- **Bajo UDP:** El impacto en descriptores y memoria es insignificante: el Broker continúa atendiendo a los 100 publicadores a través del **mismo y único socket** en el puerto 8080. No se requiere mantener estructuras de estado por conexión. El único factor crítico es la tasa agregada de datagramas: si la frecuencia de noticias de los 100 partidos satura la velocidad a la que el bucle `recvfrom()` extrae datos, el buffer de entrada del socket se desbordará y los paquetes comenzarán a descartarse silenciosamente.

4.2. 2. Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario?

- **Implicaciones en la aplicación real:** Se produce una inconsistencia de estado crítica e irreversible. El hincha queda desactualizado respecto a la realidad del partido: no se entera del gol, el marcador que visualiza en su interfaz es incorrecto y las noticias posteriores carecerán de sentido cronológico (por ejemplo, leer comentarios sobre un penalti anotado sin haberse enterado de que ocurrió un gol).

- **Por qué TCP maneja mejor este escenario:** TCP asocia a cada byte transmitido un número de secuencia continuo. Cuando el receptor recibe el segmento, retorna obligatoriamente un acuse de recibo (`ACK`). Si el paquete del gol se pierde o se corrompe en el tránsito, el receptor no genera el ACK y el emisor (Broker), al vencer el temporizador de retransmisión (RTO), reenvía automáticamente el segmento sin intervención del usuario ni de la aplicación.

4.3. 3. En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica.

La idoneidad depende de la naturaleza del dato en el seguimiento en vivo:

- Para **eventos textuales, alertas de incidencias y cambios de marcador** (el caso de esta práctica): **TCP es más adecuado**. Al ser eventos discretos e infrecuentes (un gol ocurre pocas veces por partido), la sobrecarga adicional de TCP es despreciable para la red, mientras que la confiabilidad y el orden son indispensables: un marcador deportivo no puede tolerar la pérdida de un gol.

- Para **streaming de video/audio en vivo o telemetría continua de jugadores**: **UDP es más adecuado**. En este flujo masivo de datos, la pérdida ocasional de un fotograma no arruina la transmisión, mientras que el retardo acumulativo que provocaría la retransmisión de TCP causaría congelamiento de imagen y retraso progresivo con respecto al vivo (*Head-of-Line Blocking*).

4.4. 4. Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia?

- **Comparación cuantitativa de cabeceras:**
  - En la captura UDP ([udp_pubsub.pcap]), cada datagrama incluye una cabecera fija de únicamente **8 bytes** (puertos de 16 bits, longitud de 16 bits y checksum de 16 bits). Para el mensaje `"gol"` (3 bytes), el tamaño a nivel de transporte es de solo 11 bytes.
  - En TCP, la cabecera base es de **20 bytes** (y aumenta hasta 32–60 bytes al incorporar opciones como Timestamps, SACK o Window Scale). Para los mismos 3 bytes de datos, el segmento ocupa al menos 23 bytes (más del doble de sobrecarga relativa).
- **Sobrecarga protocolar adicional:** TCP exige paquetes de control que viajan sin carga útil de aplicación: 3 paquetes para el *handshake* de conexión (SYN, SYN-ACK, ACK), paquetes adicionales de confirmación ACK para verificar los mensajes entregados, y 4 paquetes para la terminación (FIN, ACK).
- **Influencia en la eficiencia:** UDP ofrece una eficiencia de red significativamente superior en consumo de ancho de banda y ratio de datos útiles vs. bytes transmitidos, lo cual es ventajoso para transmisiones masivas y dispositivos con recursos de red limitados.

4.5. 5. Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2–1 y luego el 1–1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación?

- **Efectos en la experiencia de usuario:** Genera desconcierto, pérdida de credibilidad y errores visuales graves. El hincha ve temporalmente un marcador avanzado (2–1) que luego se rebobina.[a] un estado anterior (1–1), o lee una secuencia incongruente de eventos (ej. celebración de un gol antes de la falta que lo originó).
- **Solución a nivel de aplicación:** Se debe enriquecer la carga útil con metadatos de control:
  1. Incluir un **número de secuencia monótonamente creciente** por partido (ejemplo: `PUB:futbol:SEQ=12:gol`).
  2. Incluir una **marca de tiempo (*timestamp*)** generada por el publicador con precisión de milisegundos.
  3. En el suscriptor, se implementa una lógica de descarte o búfer de reordenamiento: cualquier mensaje cuyo número de secuencia sea menor o igual al último procesado se ignora inmediatamente, impidiendo la visualización de estados obsoletos.

4.6. 6. ¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto?

Cuando un publicador emite una noticia de un partido con N suscriptores, el Broker debe replicar el mensaje individualmente N veces:

- **Bajo UDP:** El Broker ejecuta un bucle con N llamadas a `sendto()` sobre el mismo descriptor de socket. Dado que la operación deposita inmediatamente el datagrama en el buffer de salida de la tarjeta de red sin esperar acuses, el coste computacional es lineal y mínimo. Si N crece a miles, el límite reside en la velocidad de la tarjeta de red para evacuar paquetes.
- **Bajo TCP:** El Broker debe llamar a `send()` en N sockets distintos. Cada socket mantiene su propia ventana de congestión y control de flujo. Si unos pocos suscriptores tienen redes lentas o con pérdida de paquetes, sus buffers de transmisión en el Broker se llenan. Si el Broker no utiliza sockets no bloqueantes o múltiples hilos, la lentitud de un suscriptor retrasará el envío hacia los demás, degradando el desempeño global.

4.7. 7. ¿Qué sucede si el broker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión?

- **En TCP:** La caída del Broker destruye el estado de las conexiones. Al caerse el proceso, el sistema operativo envía paquetes `RST` a los clientes que intenten transmitir, o los clientes detectan la desconexión mediante `recv() == 0`. Tanto publicadores como suscriptores quedan en estado de error y no pueden continuar hasta ejecutar un bucle de reconexión explícito que repita el *handshake* completo.
- **En UDP:** Como no existe noción de sesión a nivel de red, los publicadores y suscriptores **no se enteran** inmediatamente de la caída del Broker. El publicador continúa ejecutando `sendto()` al vacío sin recibir errores directos. Sin embargo, si el Broker se levanta nuevamente en el mismo puerto, puede recibir datagramas de inmediato sin necesidad de reconexión; no obstante, el Broker habrá perdido su tabla de suscriptores en memoria, por lo que los suscriptores no recibirán noticias hasta que vuelvan a emitir un datagrama `SUB:<tema>`.

4.8. 8. ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué?

- **Dificultad física:** En redes IP con transmisión unicast (un paquete por receptor), garantizar una llegada perfectamente idéntica en tiempo es inviable debido al jitter, las diferencias en los tiempos de ida y vuelta (RTT) y el retardo de encolamiento de cada camino de red.
- **Protocolo que mejor facilita la sincronización: UDP facilita mejor la sincronización**:
  - **En Unicast:** En UDP, el Broker despacha los N datagramas en una ráfaga casi simultánea sin detenerse a esperar acuses de recibo. En TCP, cada cliente negocia ventanas distintas y sufre retransmisiones particulares; si un cliente experimenta una pérdida, el mecanismo de control de congestión reduce su tasa y sufre un desfasaje de varios segundos con respecto a los clientes que recibieron los paquetes limpiamente.

- **En Multicast/Broadcast:** UDP permite de forma nativa la transmisión Multicast (IP clase D), donde el Broker emite un único paquete a la red y los enrutadores lo replican simultáneamente a todos los suscriptores, alcanzando la máxima sincronización posible en hardware. TCP no admite Multicast.

4.9. 9. Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró?

- **Consumo de Memoria:**
  - **TCP:** Exige una huella de memoria en el kernel sustancial. Cada socket conectado mantiene estructuras complejas de control de estado (`struct sock`, máquina de estados TCP, buffers de recepción y envío `SO_RCVBUF` y `SO_SNDBUF` que ocupan entre 64 KB y 256 KB por conexión). Gestionar 10.000 clientes TCP puede requerir varios gigabytes de RAM solo en el espacio del kernel.
  - **UDP:** Utiliza una huella de memoria insignificante. El Broker maneja un único socket en el puerto 8080 con un único buffer de entrada/salida. El registro de los clientes se mantiene en el espacio de usuario (un arreglo de estructuras `struct sockaddr_in` de apenas 16 bytes por cliente).
- **Consumo de CPU:**
  - **TCP:** Requiere una carga de CPU considerablemente mayor para procesar el *handshake*, calcular temporizadores de retransmisión (RTO), procesar ACKs acumulativos, ejecutar algoritmos de control de congestión (CUBIC/BBR) y realizar multiplexación de descriptores (cambios de contexto al iterar sobre sockets activos).
  - **UDP:** Muy bajo consumo de CPU. El kernel únicamente añade o remueve la cabecera de 8 bytes y valida un checksum de 16 bits sin calcular temporizadores ni gestionar transiciones de estado.

4.10. 10. Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una combinación de ambos? Justifique con base en lo observado en el laboratorio.

Para un sistema de escala masiva con millones de usuarios, la elección óptima es una **arquitectura híbrida por capas**:

1. **Capa de Distribución a Usuarios Finales (Edge a Cliente): TCP (WebSockets / HTTP/2) o QUIC (HTTP/3 sobre UDP):**
   - Las notificaciones de goles y marcadores deben llegar con confiabilidad garantizada y en orden estricto. La pérdida de una notificación de gol deteriora gravemente la aplicación.
   - La gran mayoría de dispositivos móviles operan detrás de cortafuegos residenciales y routers con NAT (*Network Address Translation*). Enviar datagramas UDP no solicitados hacia un cliente suele ser bloqueado por los NAT a menos que se mantengan túneles complejos de perforación de puertos (*hole punching*). TCP y WebSockets sobre el puerto 443 (HTTPS) aseguran compatibilidad universal y cifrado TLS.

2. **Capa de Ingestión y Comunicación Interna (Estadio al Broker Central): UDP / QUIC:**
    - Desde el estadio hacia la nube de procesamiento, se utiliza UDP optimizado para minimizar la latencia de ingestión de eventos y telemetría en tiempo real.
3. **Desacoplamiento mediante Red de Brokers Distribuidos (CDNs):**
    - Tal como se demostró en el laboratorio, un único Broker central colapsaría ante millones de clientes. Se debe desacoplar la carga mediante una jerarquía de brokers distribuidos (estilo Kafka o RabbitMQ) conectados a servidores perimetrales que atienden grupos de suscriptores localmente.